

Compte-rendu de TP — RAG avec Python et Groq

Sujet C — Assistant Code du Travail français • Ahmed Filali • Mai 2026

1. Objectif et démarche

L'objectif de ce TP était de construire un système RAG complet — sans LangChain ni LlamaIndex — capable de répondre à des questions de droit du travail français en s'appuyant uniquement sur des articles réels du Code du travail et en citant systématiquement leurs numéros. J'ai retenu le sujet C, qui combine un enjeu réel (assistance juridique grand public) et une exigence forte de traçabilité des sources, ce qui rend les choix techniques d'autant plus structurants.

2. Architecture retenue

Le système est découpé en deux scripts strictement séparés, conformément aux bonnes pratiques du cours.

indexation.py est exécuté une seule fois. Il charge les 34 articles du corpus JSON, applique un chunking récursif (taille 500 caractères, overlap 80), génère les embeddings avec *paraphrase-multilingual-mpnet-base-v2*, et persiste l'index FAISS et les métadonnées sur disque.

rag.py est exécuté à chaque session interactive. Il recharge l'index, embedde la question avec le même modèle, récupère les 4 chunks les plus pertinents via FAISS, construit un prompt enrichi et appelle l'API Groq (*llama-3.3-70b-versatile*) pour générer la réponse finale.

3. Choix techniques justifiés

Composant	Choix retenu	Justification
Source des données	Corpus manuel JSON (34 articles, 9 thèmes)	L'API Légifrance demande une inscription PISTE longue à obtenir ; l'option XML L
Chunking	Récursif avec overlap (500 car. / overlap 80)	Les articles sont des unités juridiques autonomes. Cette taille permet à chaque ar
Embeddings	<i>paraphrase-multilingual-mpnet-base-v2</i> (768d)	Modèle multilingue obligatoire pour le français. Bon compromis qualité/vitesse. Ve
Index	FAISS IndexFlatIP	Avec des vecteurs normalisés, le produit scalaire est égal à la similarité cosinus. S
LLM	<i>llama-3.3-70b-versatile</i> (Groq)	Modèle 70 milliards de paramètres pour la qualité de la synthèse juridique, access

4. Bonus implémentés

Bonus A — Historique de conversation. Les 6 derniers tours sont conservés et réinjectés dans le prompt, ce qui permet à l'utilisateur de poser des questions de suivi naturelles. La taille de l'historique est plafonnée afin de ne pas saturer la fenêtre de contexte du LLM.

Bonus B — Score de confiance. Si le meilleur chunk a un score cosinus inférieur à 0.35, le système refuse de répondre et invite l'utilisateur à reformuler. Cela évite que le LLM ne s'appuie sur des chunks faiblement pertinents et limite drastiquement les hallucinations sur les questions hors corpus.

Bonus C — Reformulation automatique. Avant la recherche vectorielle, un appel rapide à *llama-3.1-8b-instant* reformule la question utilisateur en formulation juridique précise. Une question familière comme « j'ai été viré, qu'est-ce que je peux faire ? » devient « Quels sont les droits du salarié en

cas de licenciement ? ». Cette étape atténue le phénomène de dilution sémantique sur les questions familières ou trop longues.

5. Difficultés rencontrées

Choix de la source de données. Le sujet propose trois options : API Légifrance (OAuth2), XML LEGI (volumineux) ou corpus manuel. J'ai d'abord exploré l'API Légifrance, mais l'inscription PISTE prend plusieurs jours pour un usage pédagogique. J'ai donc opté pour le corpus manuel JSON, ce qui m'a aussi permis de garantir l'exactitude des numéros d'articles cités — point critique pour un assistant juridique.

Score FAISS : L2 ou IP ? Le sujet ne tranche pas explicitement. Avec *IndexFlatL2*, le score est une distance (plus c'est petit, mieux c'est) ; avec *IndexFlatIP* sur des vecteurs normalisés, c'est une similarité cosinus directe (plus c'est grand, mieux c'est). J'ai choisi le second pour la lisibilité du seuil de confiance et la fidélité à la définition mathématique du cours.

Dilution sémantique sur questions familières. Lors des tests, j'ai constaté que des questions formulées de manière relâchée (« combien j'ai dans le congé payer ») produisaient des chunks de moins bonne qualité que des questions juridiques précises. Le bonus C de reformulation par LLM règle ce problème de manière élégante en uniformisant la formulation avant l'embedding.

Modèles Groq dépréciés. Au cours du projet, j'ai dû mettre à jour les noms des modèles : *llama3-8b-8192* a été déprécié au profit de *llama-3.1-8b-instant*, et *llama3-70b-8192* au profit de *llama-3.3-70b-versatile*. C'est un rappel utile que les API LLM évoluent rapidement et qu'il faut surveiller la documentation officielle.

Prompt engineering du LLM. Forcer le LLM à respecter strictement le format « réponse + citation d'article + avertissement juridique » a demandé plusieurs itérations. La clé a été de numérotter explicitement les 7 règles dans le prompt système et d'imposer la mention d'avertissement au format exact attendu.

6. Décisions de conception

Stack minimaliste. Aucun framework lourd : seulement *groq*, *sentence-transformers*, *faiss-cpu*, *numpy* et *python-dotenv*. L'ensemble du code tient en moins de 400 lignes Python clairement commentées.

Texte enrichi à l'embedding. Le numéro d'article et le titre sont inclus dans le texte embedded, et pas seulement en métadonnée. Cela permet à la recherche de capter des requêtes du type « quelle indemnité de licenciement ? » même quand le mot-clé apparaît dans le titre et non dans le corps.

Avertissement juridique imposé par le prompt. La règle 5 du prompt système oblige le LLM à terminer chaque réponse par la mention exacte recommandée par le sujet. Ainsi, l'avertissement ne peut jamais être oublié, quel que soit le ton ou la formulation de la question.

Gestion d'erreurs robuste. Le script *rag.py* gère explicitement l'absence de clé API, l'absence d'index, les erreurs réseau de l'API Groq et les interruptions clavier. Le bonus C dispose d'un fallback sur la question originale si la reformulation échoue.

7. Évolutions envisagées en production

Pour une mise en production, j'ajouterais : un **reranker** (Cohere Rerank ou un cross-encoder local) pour passer de top-20 à top-4 avec une qualité supérieure ; une migration vers **Qdrant** ou ChromaDB pour gérer un corpus complet du Code du travail (environ 10 000 articles) ; une **API REST FastAPI** à la place de la CLI ; un **monitoring** des scores et des questions hors couverture ; et un système d'évaluation automatique avec **Ragas** pour mesurer la *faithfulness*, l'*answer relevance* et la *context precision*.

Le code source complet, le README détaillé et le corpus de 34 articles accompagnent ce compte-rendu dans le dossier de rendu.